

To call a method, give the name of the instance (in this case, `my_dog`) and the method you want to call, separated by a dot. When Python reads `my_dog.sit()`, it looks for the method `sit()` in the class `Dog` and runs that code. Python interprets the line `my_dog.roll_over()` in the same way.

Now Willie does what we tell him to:

```
Willie is now sitting.  
Willie rolled over!
```

This syntax is quite useful. When attributes and methods have been given appropriately descriptive names like `name`, `age`, `sit()`, and `roll_over()`, we can easily infer what a block of code, even one we've never seen before, is supposed to do.

Creating Multiple Instances

You can create as many instances from a class as you need. Let's create a second dog called `your_dog`:

```
class Dog:  
    --snip--  
  
my_dog = Dog('Willie', 6)  
your_dog = Dog('Lucy', 3)  
  
print(f"My dog's name is {my_dog.name}.")  
print(f"My dog is {my_dog.age} years old.")  
my_dog.sit()  
  
print(f"\nYour dog's name is {your_dog.name}.")  
print(f"Your dog is {your_dog.age} years old.")  
your_dog.sit()
```

In this example we create a dog named Willie and a dog named Lucy. Each dog is a separate instance with its own set of attributes, capable of the same set of actions:

```
My dog's name is Willie.  
My dog is 6 years old.  
Willie is now sitting.  
  
Your dog's name is Lucy.  
Your dog is 3 years old.  
Lucy is now sitting.
```

Even if we used the same name and age for the second dog, Python would still create a separate instance from the `Dog` class. You can make as many instances from one class as you need, as long as you give each instance a unique variable name or it occupies a unique spot in a list or dictionary.